CSC-302: Programming in Python

Unit-III: **File Handling**



Renaming and Deleting Files

- In Python, you can rename and delete files using built-in functions from the **`os`** module.



Renaming files

- To rename a file in Python, you can use the **`os.rename()`** function
- This function takes essentially **two arguments**: **path of the file to be renamed** and **the new filename**
- Syntax: **`os.rename(path, new_filename)`**



- Usage example

```
from os import rename

new_filename = input('enter new filename:')
rename('./test.txt', new_filename)
```

  - Above code will rename **`test.txt`** in the current directory to whatever is **`new_filename`**



- If the file does not exist, Python will generate a **`FileNotFoundError`**
- If the file exists, but exist in a directory with no write permission, Python will generate a **`PermissionError`**



Deleting files

- We can delete a file in python using **`os.remove()`** function
- This function deletes a file specified by its path
- Syntax: **`os.remove(path)`**



- Usage example

```
from os import remove

remove('./something.txt')
```

  - Above code deletes the file named **`something.txt`** in the current working directory



- Same thing as rename, **`FileNotFoundError`** if file doesn't exist
- And **`PermissionError`** if the parent directory is protected



Changing file pointer position

- This method is used to **position the file object at a particular position** in a file
- Syntax: **`file_object.seek(offset [, reference point])`**
  - **`offset`** is the number of bytes by which the file object is to be moved
  - **`reference_point`** is the starting position of the file object
    - **`0`** for beginning of the file, **`1`** for current position and **`2`** for end of file. The default value is **`0`**
    - files opened in text mode only supports **`reference_point 0`**



- Usage example

```
f = open('./lyrics.txt', 'r')

f.seek(8)
college_name = f.read(10)
print('the anthem is for', college_name)

f.close()
```

lyrics.txt

```
sec for st.edmunds,
winners in every field.
cheer all cheer for st.edmunds,
heroes who never yield!
```

Output: the anthem is for st.edmunds



- If a negative integer is provided, it will result in a **`ValueError`**
- If a non-integer number is provided, it will result in a **`TypeError`**



File methods - `fileno()`

- The Python function **`fileno()`** method **returns the file descriptor (or file handle) of an open file**
- A file descriptor is an **unsigned integer that is used by the operating system to identify the open files in an os kernel**
- Syntax: **`fileObject.fileno()`**

  - **params**: the method doesn't require any parameters

  - **return**: the method returns the integer file descriptor



- Usage example

```
f = open('lyrics.txt', 'r')
fid = f.fileno()
print('the file descriptor of', 'lyrics.txt = ', fid)
f.close()
```

  - **Output**: `the file descriptor of lyrics.txt = 3`



- Every process gets three file descriptors automatically
  - **`0: stdin`**
  - **`1: stdout`**
  - **`2: stderr`**
- The very first file you open gets the next available number, which is almost always **`3`**



File methods - `flush()`

- **Whenever something is written to a file intiaillly it is written to the internal buffer**; which is **then transferred into the destination file once the buffer is full**
- This is done **to prevent frequent system calls for every write operation**
- **Once the file is closed, Python automatically *flushes* the buffer**



- The Python File **`flush()`** method **flushes the internal buffer explicitly, without needing it to be full**
- Syntax: **`file_object.flush()`**
  - **params**: the method doesn't require any parameters
  - **returns**: the method doesn't return any value or **`None`**



- Usage example

```
f = open('lyrics.txt', 'a')

f.write('\nheroes who never yield!')
f.flush()

f.close()
```



File methods - `isatty()`

- **`isatty()`** method **checks whether the file stream is interactive or not**
- The file stream is said to be interactive **if the file is connected to the "tty" type device**
- **"tty" is an abbreviation for "*Teletypewriter*"** and **refers to an interactive terminal device**



- Syntax: **`file_object.isatty()`**
  - **params**: The method doesn't accept any parameters
  - **returns**: The method returns a Boolean value
- Usage example

```
f = open('/dev/tty')
print(f.isatty())    # True
f.close()
```



File methods - `truncate()`

- **`truncate()`** method **truncates or shortens a file's size**
- **`truncate(size)` sets the file to exactly `size` bytes. If the file was larger, the extra data is cut off and lost. If the file was smaller, the new space is filled with null bytes (`\x00`).**
- The **default value of this argument is the current file position**



- Syntax: **`file_object.truncate(size)`**
  - **params**: **`size`**: optional, The size at which a file is to be truncated
  - **return**: The method doesn't return anything or None



- Usage example

```
sec for st.edmunds,
winners in every field.
cheer all cheer for st.edmunds,
heroes who never yield!
```

lyrics2.txt (99 bytes)



```
f = open('./lyrics2.txt', 'a')
f.truncate(50)
f.close()
```



```
sec for st.edmunds,
winners in every field.
cheer
```

lyrics2.txt (50 bytes)





Directory methods

- **Directory is a collection of files of the same or different formats**
- Python has a variety of built-in functions in the **`os`** module to manage directories
- Managing directories involve creating, removing and changing directories



Directory methods – `mkdir()`

- This function is used to **create directories in the path provided as the argument**
- The minimal syntax is as **`os.mkdir(path)`**
- Usage example

```
import os

os.mkdir('./CollegeFiles')
print('Directory created successfully')
```

- The above code creates the directory in the current working directory





Directory methods – `getcwd()`

- This function is used to **display the current working directory**
- Syntax: **`os.getcwd()`**
  - **params**: This function doesn't take any arguments
  - **returns**: This function returns a string



- Usage example

```
import os

print(os.getcwd())     # /home/abanggi/py
```

- **Output:** `/home/abanggi/python/files`



Directory methods – `chdir()`

- The **`chdir()`** function is used to change the current directory
- Syntax: **`os.chdir(path)`**
  - **params**: The function takes the path of the new directory
  - **returns**: Thee function doesn’t return anything or None
- A **`WindowError`** is generated if the path specified doesn't exist



- Usage example

```
import os

print('current working directory', os.getcwd())
# output → /home/abanggi/python/files

os.chdir('/home/abanggi/Documents')

print('current working directory', os.getcwd())
# output → /home/abanggi/Documents
```



Directory methods – `rmdir()`

- The **`rmdir()`** function is used **to remove or delete a directory**
- Syntax: **`os.rmdir(path)`**
  - **params**: The function takes the **path** of the target directory
  - **returns**: The function doesn't return anything or None
- **This function can only delete directories which are empty**



- Usage example

```
import os

os.rmdir('./TestDirectory')
print('Directory removed successfully!')
```

- If the directory is not empty it generates an **`OSError`** exception

